Ton Duc Thang University
Faculty of Information Technology

FINAL PROJECT

Course: Introduction to Artificial Intelligence

Duration: 06 weeks

I. Formation

1. The project is conducted in groups of students.
2. Student groups conduct required tasks and submit the project following instructions below.

II. Tasks

a) Task 1 (3.0 point(s)): Local Search & Propositional Logic

Descriptions: N-queen problem

- Given a chess board with the size of N x N, students implement a solver to find a solution for placing N queens on the board without attacking pairs.
- Assign to each square on the board a logical variable in which if the variable holds the value True, the square contains a queen, and if the variable holds the value False, the square is empty.

Requirements:

1. *(1.0 point(s))*: Formulate the given problem as a Propositional Logic problem and introduce its relevant components. Discuss an algorithm for discovering the constraints that govern the placement of queens on the chessboard, and explain the procedure for converting these constraints into Conjunctive Normal Form (CNF) clauses.
2. *(1.0 point(s))*: Use the Glucose3 solver from the PySAT library to find a solution to the given problem and conduct experiments with increasing values of N, where N >= 4. For each value of N, report the total number of CNF clauses and the running time required by the solver.
3. *(1.0 point(s)):* Solve the problem using the Min-Conflicts algorithm introduced in Lecture 08 and propose a heuristic for selecting variables. Conduct

experiments to evaluate the effectiveness of the proposed heuristic by comparing its performance with the standard Min-Conflicts approach.

Notice:

- Recommended editor: Visual Studio Code
- The project must be designed and implemented following Object-Oriented Programming principles.
- The evaluation configurations: macOS 13.7.8 (Ventura) with an Intel Core i5 processor.

b) Task 2 (3.0 point(s)): Adversarial Search

Descriptions: Tank Battle Game

- The game is designed as a turn-based game. During each game cycle, players take actions and the game state is updated accordingly.
- Each tank can move one cell per turn (UP, DOWN, LEFT, RIGHT). When a tank fires a bullet, the bullet travels at a speed of 3 cells per game cycle in the direction in which the tank is facing.
- The bullet movement is processed cell by cell. For each game cycle, the bullet moves up to three cells, and a collision check is performed after each cell movement. If the bullet encounters a wall, tank, or base, the corresponding collision event is triggered immediately. A bullet stops immediately after hitting an entity.
- Each game entity has a predefined amount of health as in the table.

Entity	Initial HP	Damage per Hit
Tank	6	1
Base	9	1
Wall	3	1

- A tank entity has 06 actions, including MOVE_UP, MOVE_DOWN, MOVE_LEFT, MOVE_RIGHT, FIRE, STAY.
- The game ends when either a base or a tank is destroyed. The player whose tank and/or base remains operational is declared the winner.

- Students are required to apply adversarial search algorithms introduced in Lecture 07, such as Minimax Search, Alpha-Beta Pruning, and Heuristic Minimax Search, to control the tank.



(https://www.hardcoregaming101.net/tank-battalion-battle-city/)

Requirements:

1. *(1.0 point(s)):* Formulate the given problem as a Zero-Sum Game and introduce its relevant components, including the game state, players, actions, transition function, terminal states, and utility function. Discuss how adversarial search algorithms, such as Minimax Search, Alpha-Beta Pruning, and Heuristic Minimax Search, can be applied to control the tanks.
2. *(1.5 (point(s)):* Implement an algorithm to control the tanks in the game. Students may apply various optimization techniques to ensure that the tanks operates efficiently and the game runs in real time. Set up experiments to compare the computational time and search performance of the following four configurations: Minimax Search, Alpha-Beta Search, Heuristic Minimax Search, and Heuristic Minimax Search with Alpha-Beta Pruning. The experiments should use the same game scenarios and search conditions where applicable to ensure a fair comparison. Relevant metrics may include execution time, number of explored game states, search depth, and other appropriate performance measures.

3. *(0.5 point(s))*: Implement a graphical user interface using the PyGame library to operate the game. The system should provide a mechanism for importing and integrating algorithms developed by other student groups, allowing different algorithm implementations to compete against each other under the same game environment.

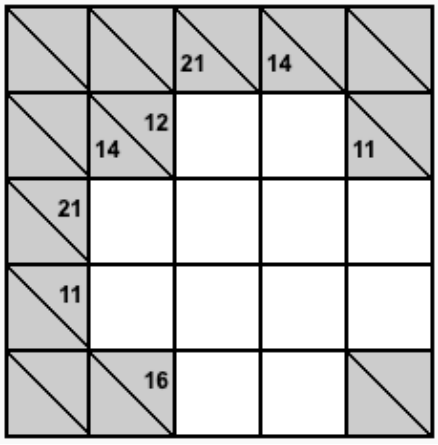
Notice:

- Recommended editor: Visual Studio Code.
- The project must be designed and implemented following Object-Oriented Programming principles.
- The evaluation configurations: macOS 13.7.8 (Ventura) with an Intel Core i5 processor.

c) Task 3 (3.0 point(s)): Constraint Satisfaction Problems

Descriptions: Kakuro

- Students are expected to familiarize themselves with the rules and gameplay of Kakuro by referring to the Kakuro article on Wikipedia (https://en.wikipedia.org/wiki/Kakuro).
- Given the example Kakuro game board shown below, students are required to formulate the game as a Constraint Satisfaction Problem (CSP) by identifying the variables, their corresponding domains, and the constraints governing the puzzle.

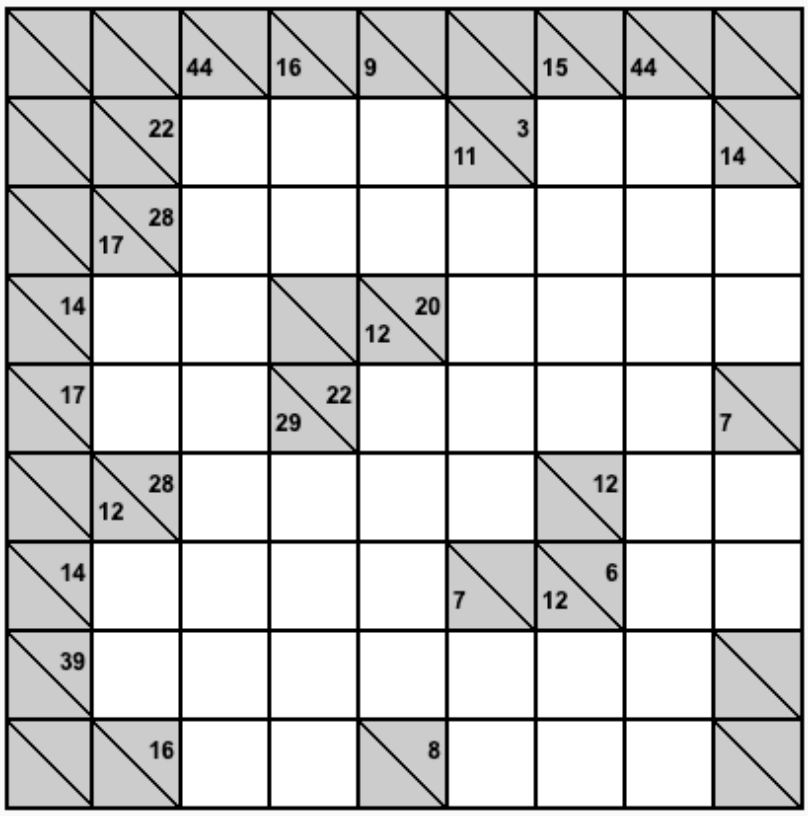


- Students should then develop a solver that combines constraint propagation techniques with Backtracking Search to find a valid solution. The

implementation should incorporate appropriate heuristic techniques introduced in Lecture 08 to improve the efficiency of the search process.

Requirements:

1. *(1.0 point(s)):* Formulate the given problem correctly as a Constraint Satisfaction Problem (CSP), clearly identifying and explaining the relevant components, including variables, domains, and constraints. Describe and justify the algorithms and techniques applied in the project.
2. *(1.0 point(s)):* Implement a solver capable of finding a valid solution to the given problem. Demonstrate and evaluate the solver's correctness, stability, and effectiveness by applying it to the more complex test case provided below.



3. *(1.0 point(s)):* Implement a user interface (UI) for the game that clearly visualizes the domains of variables, the constraint propagation process, and the Backtracking Search process.

Notice:

- Recommended editor: Visual Studio Code.

- The project must be designed and implemented following Object-Oriented Programming principles.
- The evaluation configurations: macOS 13.7.8 (Ventura) with an Intel Core i5 processor.

d) Task 4 (1.0 point(s)): Report and Interview

- Student groups compose the project report using the IEEE conference proceeding template.
- Recommended editor: Overleaf.
- Selective contents:
  - *Title*: the project title
  - *Authors*: group member's information, the lecturer is appended as the last author.
  - *Abstract*: summarize the project requirements, approaches, experimental results, and levels of completion.
  - Each following section presents a task in the project, with a meaningful and human-readable title. Briefly introduce the approach to tackle the problem and illustrate results with related figures/tables, etc.
  - "*Contributions*" section: individual tasks, individual completion levels (0%-100%).
  - "*Self-evaluation*" section: self-evaluate task completion and estimate scores.
  - "*Conclusion*" section: summarize the project requirements, approaches, experimental results, and levels of completion.
- References are in the IEEE format.
- Maximal length is **05** pages in 2-column layout.
- Students must take an interview to justify your work in the project. If you can not explain clearly and exactly ideas nor implementations, then you get 0.0 points for related tasks.

III. Submission

- Create a folder whose name is in the format

  AI_final_<Project Group ID>_<Student ID>

  - **source:** source code files, each task is located in a subfolder
    - REAME.md and requirement.txt are required to set up projects.
  - **report.pdf:** report of the project
  - **demo.txt**: a URL to the demo video.
- Students maintain outputs of all cells in ipynb files.
- Compress the folder into a zip file and submit by the deadline.
- **Every team member must submit the project individually**.

IV. Policy

- **Student groups that submit their projects late will receive 0.0 points for each group member.**
- **Missing any required materials from the submission will result in a deduction of at least 50% of the presentation score.**
- **Copying source code from the Internet or other students, sharing your work with other groups, or engaging in similar forms of academic misconduct will result in a score of 0.0 for all groups involved.**
- **If there are any indications of unauthorized copying or sharing of the assignment, additional interviews may be conducted to verify the authenticity of the students' work.**
- **The use of AI tools is prohibited in this project. If any AI-generated symbols or other identifiable AI-generated content are found in the source code files, the group will receive a score of 0.0 for the corresponding task.**

-- THE END --